

ARIA Airline Chatbot — Botpress AI Customer Support System

Cleaned Portfolio Documentation

Focus Areas:

- Botpress chatbot workflow design**
- Airline customer support automation**
- Knowledge base integration**
- Semantic network knowledge representation**
- Booking, baggage, flight status, and travel policy support**
- Webchat / website integration**

Prepared by: Abdulaziz Qaderi

Table of Contents

1.	Introduction	3
2.	Methodology	4
2.1	Design approach for Aria	4
2.2	Menu-first conversation architecture.....	4
2.3	Planning artefacts created during the design phase.....	5
2.4	Flowchart diagrams	5
2.5	Interaction safeguards and quality controls built into the design	9
2.6	User stories used to define scope and validate completeness.....	9
2.7	Knowledge Representation.....	10
2.8	Tools and technologies used	11
2.9	Development Process.....	11
3.	Implementation	12
3.1	Challenges Faced.....	12
3.2	Solutions to Challenges	12
3.3	Key Functions Implemented	12
3.4	Demonstration of Chatbot Features and Deployment.....	13
3.5	Inputs and Outputs	13
3.6	Customer Navigation and Chatbot Features	21
3.7	Workflow Structure	21
3.8	Knowledge Bases Used.....	21
3.9	Website Integration for Professional Presentation	23
4.	Conclusion.....	24
5.	References	25

This project was developed as part of an Introduction to Artificial Intelligence academic group project. The complete chatbot was built collaboratively to simulate an airline customer support assistant. This cleaned portfolio documentation focuses on the chatbot design, **Botpress** implementation, knowledge base structure, data tables, and demonstration materials prepared for public portfolio use.

1. Introduction

The airline customer service chatbot works as a virtual assistant who uses advanced technology to support travelers through all stages of their journey. The system offers customers fast automated responses that deliver both accuracy and speed for their standard inquiries about flight planning and ticket modifications and check-in procedures and luggage information and flight status and travel policy questions. The airline uses this system to create better customer communication through its digital platform, which provides customers with real-time assistance. The chatbot serves as the initial contact point for passengers who need help with basic requests, while the system will guide them through their travel requirements from start to finish.

The airline company requires this chatbot because customer inquiries have increased and customers need continuous support throughout the day. Customers who use traditional service channels experience service delays because the system receives excessive call volume, which results in longer wait times and higher operational expenses. The airline will achieve three benefits through chatbot implementation, which include decreased human agent work requirements and improved response times and steady service delivery. The project provides advantages, which include better customer satisfaction and lower operational expenses and greater operational efficiency and better resource distribution and a digital experience, which helps the airline maintain its competitive advantage.

1.1 Group Project Context and My Contribution

This project was developed as part of an Introduction to Artificial Intelligence academic group project. The complete ARIA chatbot was built collaboratively to simulate an airline customer support assistant for passengers. The overall system includes customer intake, main menu routing, flight status checking, booking management, baggage support, travel policy assistance, human-agent escalation, knowledge base integration, structured Botpress tables, and website/webchat deployment. My contribution focused on the overall chatbot design and presentation of the system, especially the knowledge representation and implementation structure. I worked on documenting how ARIA uses semantic networks to represent airline-related entities such as passengers, bookings, tickets, flights, payments, baggage, airports, terminals, and gates. I also contributed to explaining how the Botpress knowledge base, structured tables, AI transitions, and workflow branches support the chatbot's customer service functions.

In addition, I helped prepare the portfolio-ready documentation by organizing the chatbot's methodology, knowledge base structure, Botpress implementation evidence, screenshots, flowcharts, and demonstration materials. This cleaned version focuses on presenting the project professionally for GitHub and LinkedIn while keeping the original group project context clear.

2. Methodology

2.1 Design approach for Aria

Aria was conceived as a dependable assistant for passengers with common airline enquiries. The planning phase of the project included producing preliminary sketches, flowcharts, and user stories to inform the design of the chatbot before any implementation occurred. The selected chatbot platform was a low-code, no-code chatbot platform appropriate for the creation of a rule-based chatbot with no machine learning capabilities. The visual modeling of the conversation within the platform best matched the rule-based structure of the airline chatbot's conversations, enabling the developer to implement the chatbot's functions without requiring training data with the airline's tasks. The overall design strategy for the chatbot incorporates elements of human-centered design (HCD) and iterative prototyping. Human-centered design principles influenced the structure of Aria's conversations. For example, the chatbot does not allow the user to type unrestricted text into the chat application to minimize the cognitive load on the users and to prevent the users from becoming uncertain as to how to interact with the chatbot.

According to human-centered design principles (ISO, 2019), the system should be structured in a way that minimizes the effort required of the users to accomplish common tasks. Therefore, the most common tasks performed by airline passengers (checking flight status, checking baggage policy, booking flights) are made accessible from the main menu of the chatbot to minimize the number of steps the users must take to access these tasks. Furthermore, Aria's main menu makes it easy for the users to access the most common tasks of the chatbot and ensures that the users are not likely to become trapped in one specific stage of the chatbot's interactions with the user. This reflects human-centered design elements of making sure that the system prevents avoidable errors by the users (Laubheimer, 2016). Another reason for selecting a rule-based conversation structure for Aria instead of a natural language processing chatbot is because airline tasks are inherently structured and goal-oriented.

Using a rule-based system for determining the airline chatbot's conversations reduces the chance of misinterpreting the user's input and increasing the likelihood that the chatbot will accurately fulfill the users' tasks.

2.2 Menu-first conversation architecture

Aria implemented a menu-first structure that allows the user to select the type of information they would like to obtain from the chatbot at the start of their conversation (for example, Flight Status or Booking). This structure allows for the users to more easily communicate with the chatbot with less ambiguity in the beginning of their conversation (Quynh N., 2022).

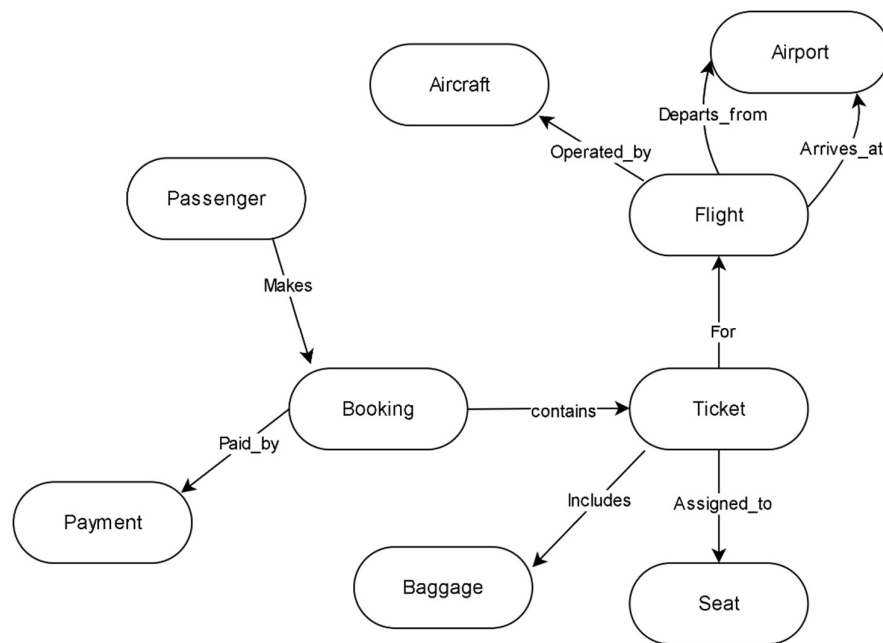
Furthermore, Aria uses a form and slot-filling structure within each menu selection, asking the user for certain information that is required to provide the information that they requested (such as the date of the flight, and its identifier). The use of this structure allows for the chatbot to more easily fulfill the user's request, as slot filling is often used by chatbots to acquire the information that is necessary to fulfill the user's intent (Mauajama Firdaus, 2019). Each of the parameters required of the user (such as the date of the flight, the booking reference number, or the identity of the passenger) are entities that are required to have a value in order for the chatbot to fulfill the user's request. Thus, the chatbot maintains a dialogue state that requires these entities to be filled out, and does not begin to fulfill the request until these entities are gathered from the user.

2.3 Planning artefacts created during the design phase

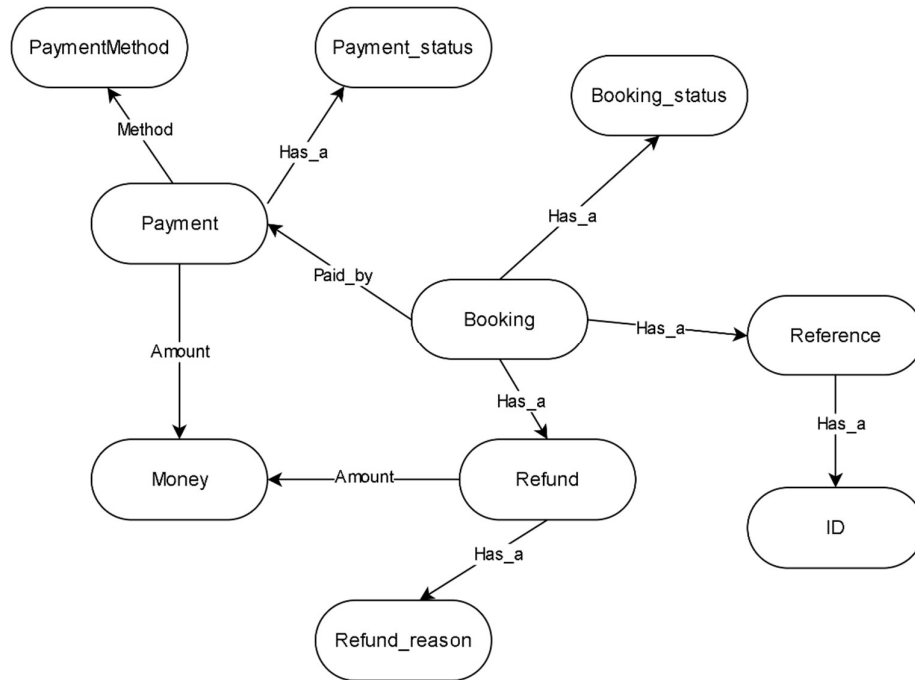
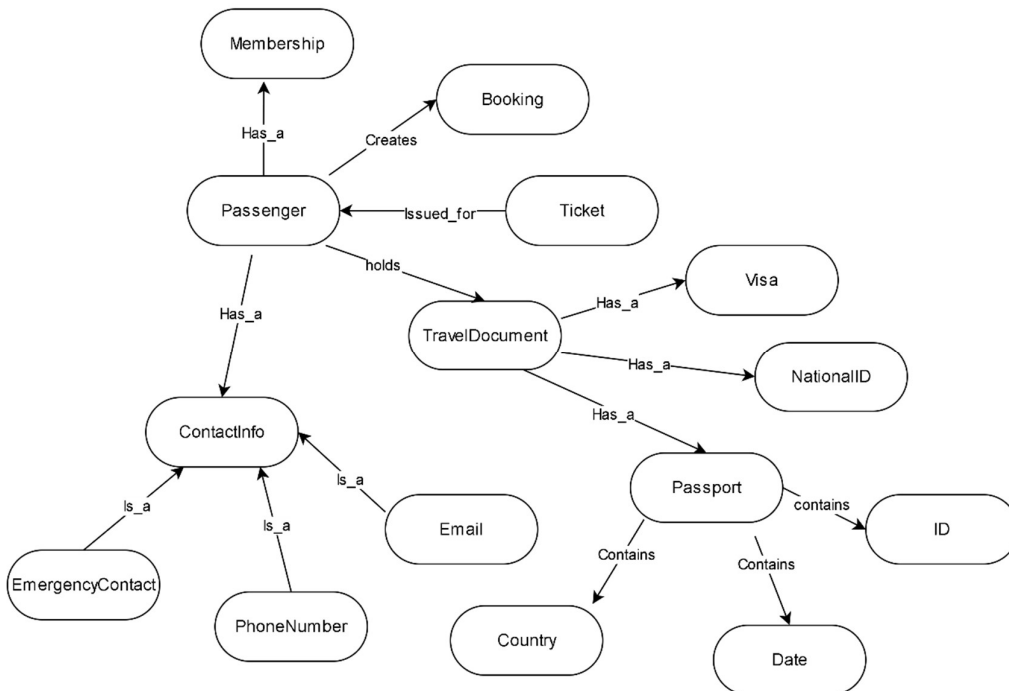
During the planning phase, the developers created rough sketches of the conversation and screen layouts to determine how the users would find the necessary functions once they interacted with the airline. The main deliverable from the planning phase was a series of flowcharts that represented the conversation that should occur between the airline and each of its various services (Flight Status, Booking, Baggage, Airport/Terminal information, Travel guidelines, and Other requests). These flowcharts were used to plan the user interaction with the chatbot and to ensure that each of the users' paths within the chatbot would have a valid endpoint for their interactions, as represented by the research in creating UML activity diagrams to represent and test the use cases for a product or service (Nayak & Samanta, 2011).

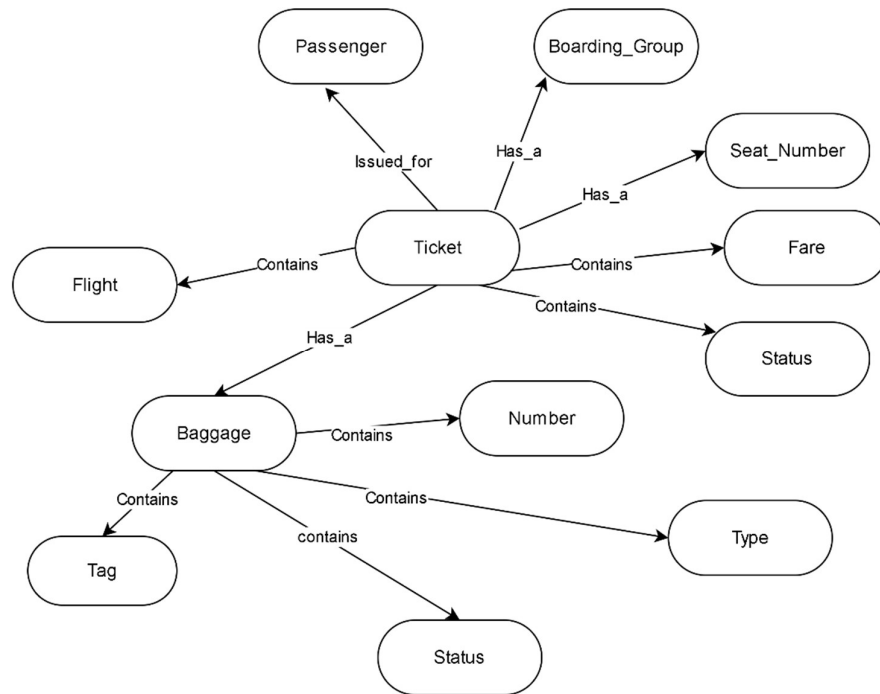
In addition to these flowcharts, they were also utilized as the dialogue state model for the chatbot. Each box within the flowchart represented a stage within the conversation between the user and the airline's chatbot, and each arrow represented the specific way in which the users were to interact with the chatbot. By utilizing these flowcharts as the dialogue state model, the chatbot ensures that it cannot be directed to perform actions in an incorrect order by the user, and that the chatbot is, therefore, a rule-based chatbot rather than a natural language interaction chatbot.

2.4 Flowchart diagrams

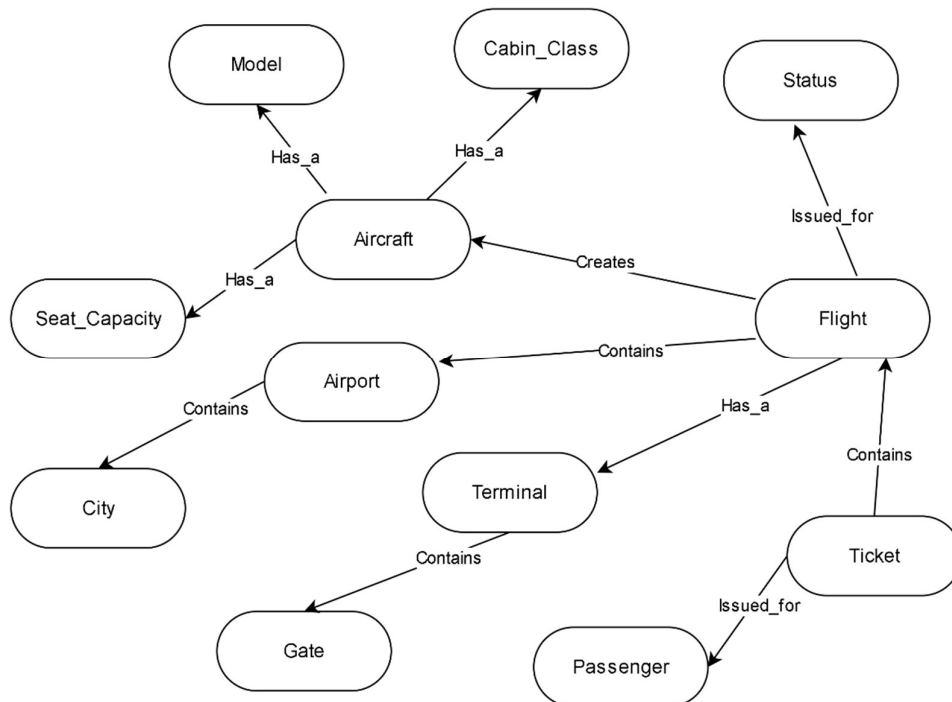


Fig(1)

**Fig(2)****Fig(3)**



Fig(4)



Fig(5)

The following conceptual diagrams display the various states and the possible transitions between those states that the chatbot can utilize to navigate the conversation regarding airline information. The diagrams indicate the states through which the chatbot will move during the conversation rather than utilizing a rigid script for the chatbot to follow. As indicated in the diagram in Fig. 1, the main states of

the chatbot are Passenger, Booking, Ticket, and Flight; however, there are also sub-states for categories like Seat, Baggage, Payment, Aircraft, and Airport. These states form the primary means of navigation for the chatbot in order to determine the main topic of the customer's conversation with the chatbot. During testing of the diagram, it became apparent that customers often wished to access information related to flights yet did not wish to first navigate to their booking or ticket. To account for this behavior, a constraint was placed upon the customers that they must first select a category of service from the main navigation menu before selecting information related to that service; this constraint is reflected in the implemented chatbot interface.

The transaction logic is presented in more detail in Fig. 2. Within this example, the transaction includes information about the **payment method, amount, status, booking status, reference ID**, and even the refund details for that booking. Thus, the chatbot can handle any of these types of enquiries by accessing the appropriate status nodes associated with that booking. Furthermore, the transaction also displays the relationships among the system entities (such as how the refund status depends upon the booking being in a confirmed status). Finally, based upon this information, validation rules were placed within the chatbot that requires the user to provide the booking reference prior to requesting the payment or refund information. Thus, the chatbot can eliminate any invalid conversations regarding the payment or refund.

User identity handling is represented in Fig. 3. The passenger node is connected to contact information, membership, and travel documents, including a passport, visa, national ID, and the ticket that is issued to the passenger. These attributes allow the chatbot to verify and modify passenger records before performing booking actions. The identity diagram also made clear that the passenger's personal data must be verified before any ticket modification actions can be performed. Thus, before the chatbot allows any changes to a passenger's ticket booking, their identity must first be verified. This requirement mirrors the constraints of the airline booking system and ensures that the chatbot's dialogue follows the constraints of the airline booking system.

Ticket content is further expanded in Fig. 4. In this ticket, information like the **boarding group, seat number, fare, and ticket status** is displayed. Furthermore, the ticket is also connected to a collection of **baggage records, each having information like tag, type, and tracking status**. Through these relationships between the ticket and baggage entities, the chatbot can respond to user questions regarding boarding, seating, tickets, and baggage tracking. Furthermore, these relationships also helped to determine the order of questions that the chatbot should ask of the user. For instance, the chatbot should first retrieve information regarding the ticket before asking any questions regarding baggage tracking. Thus, the chatbot can ensure that it has all the information that it needs to provide a complete response to the user. The operational travel information is covered in Fig. 5, which describes the connection between the **flight**, the **aircraft**, and the **airport** (specifically, the **terminal** and **gate** within the airport) where the flight originates from, and the tickets are issued to the passengers that are on that flight. Thus, the chatbot can provide information regarding the status of the flight, from which airport it is departing, and the gate where the passengers will find the flight within the airport. Such a structure for the conversation established a means of retrieving information in a hierarchical manner (from flight to airport to terminal to gate). Thus, the chatbot will always begin by asking for the flight of the passenger that is requesting the information.

Beyond providing an overview of the functionality of the chatbot, these flowcharts also acted as a specification for the behavior of the chatbot. The flowcharts were intended to reflect the conversation between the user and the chatbot, and ensured that each user interaction with the system would end in

some valid end state. Thus, the flowcharts ensured that the system would behave in a consistent manner when used by passengers.

2.5 Interaction safeguards and quality controls built into the design

Aria incorporates recovery paths and “safe exits” (such as the ability to navigate back to the main menu) that prevent users from becoming stuck in conversations with the chatbot. Any necessary prompts to provide additional information or to correct information that was provided to the chatbot indicate that the system is designed to account for the fact that users may not provide all of the necessary information or may provide incorrect information during their interactions with Aria. Furthermore, these “safeguards” also allow for the testing of Aria’s abilities to account for incorrect and incomplete inputs from users. Thus, Aria is known to be robust before it is released for public use.

2.6 User stories used to define scope and validate completeness

To make the design testable, the system requirements were written as user stories. In agile development, user stories are usually written as natural language requirements from the user’s perspective. The Connextra template asks developers to write the user story as: “As a ⟨role⟩, I want ⟨goal⟩, so that ⟨benefit⟩” (Amna & Poels, 2022).

For the Aria project, the user stories ensured that every menu option had a goal associated with it and that every step of the conversation was accounted for before development (Amna & Poels, 2022).

Examples of the user stories that guided the development of the Aria project include:

“As a passenger, I want to be able to check the status of my flight using my flight date and flight identifier so that I can discover if my departure time has changed”.

“As a passenger, I want to be able to start a new flight booking or manage an existing booking so that I can make changes without having to contact customer support”.

“As a passenger, I want to know information regarding my baggage allowance and what issues I may encounter with my baggage so that I can either account for this before traveling or resolve the issue faster”

Each user story was accounted for in the conversation flows to ensure that each user goal would reach a valid end state for the conversation flow. Any user goal that could not reach a valid end state for the conversation flow were revised before being implemented into the application.

2.7 Knowledge Representation

Semantic Networks

Semantic Networks were the primary choice of knowledge representation in the development of the Aria chatbot. A semantic network is a directed graph that represents knowledge by using nodes to represent things or concepts (e.g., Passenger, Booking, Flight, Payment) and edges to represent the relationship between things or concepts (e.g., makes, contains, paid by, departs from). The semantic network allows the system to represent hierarchical and associative relationships in the airline domain (Peters, 2003).

This choice of Semantic Networks as opposed to other approaches such as Frames or Production Systems was due to a number of reasons. To begin with, semantic networks explicitly demonstrate the associations between things. In the field of an airline, there are numerous relationships. As an example, a Passenger books a Flight, a Booking has a Ticket, and a Ticket has a Flight. These relations are easily represented by linking nodes. This simplifies the design, understanding, as well as the maintenance of the network.

Second, semantic networks facilitate inheritance of the property. As an example, a Passport is a type of Travel Document. A Travel Document that is said to hold also says the same thing about a Passport. This will remove redundancy and makes the knowledge base very consistent.

Third, semantic networks provide efficient navigation of a path. Depending on the query of a user, such as, What is my departure gate?, the chatbot can follow a relationship route: Ticket Flight Airport Terminal Gate. This feature of moving through a sequence of connected nodes assists the chatbot in making contextual reasoning and solving questions in the right way.

Comparison with Other Techniques

Frames and Production Rules

Frames and Production Rules are among the most powerful techniques of knowledge representation, but they were not chosen as the primary technique to be applied in this chatbot. Frames are helpful in describing structured objects with attributes. However, they are not very flexible in illustrating complex relationships between a number of objects.

IF- THEN Rules

Rules Production Rules (IF-THEN Rules) are relevant to decision logic. However, they can become complex and difficult to manage if the system involves a considerable number of interrelated elements such as booking, payment, identity, and flight details. Semantic networks, on the other hand, are more natural and graphical in illustrating interrelated airline processes (Russell and Norvig, 2022).

Chatbot Functionality Contribution

The semantic nets improve the functionality of the chatbot in the following ways. It enables a contextual processing, dynamic query processing, and reduces redundancy of data through inheritance. The more regular relationships also make it easier to integrate Natural Language Understanding (NLU) because the question that the users ask can be related to both nodes and relationships in the network. In general,

Semantic Networks provide a flexible, expandable, and logically organized foundation for the Aria chatbot that can ensure the correct recovery of information and logical thinking (Holdsworth, 2025).

2.8 Tools and technologies used

To achieve the project objectives and ensure a smooth development process, the team utilized a set of effective tools for design, implementation, and collaboration:

- **Draw.io:** This tool was the primary instrument for the design phase. We used Draw.io to construct the visual framework of the chatbot, creating detailed flowcharts and decision trees. This allowed the team to visualize the conversation logic and "Knowledge Representation" rules before actual implementation.
- **Botpress:** This low-code development platform was selected for the implementation phase. It enabled the team to translate the visual designs into a functional chatbot without requiring extensive hard-coding, facilitating features like automated replies and menu navigation.
- **Microsoft Teams & WhatsApp:** Used for real-time communication, file sharing, and conducting weekly progress meetings.

2.9 Development Process

The team used a structured approach to Knowledge Engineering in the formation of the chatbot. The typical methodology includes five phases – identification, conceptualization, formalization, implementation and testing (Waterman, 1986). Each of these standard phases ensured the knowledge base (Semantic Networks) had a logical structure prior to deployment. The steps were as follows:

1. **Identification of Nodes** The process began with the identification phase during which the problem and concepts were defined. Russell and Norvig (2021) state this is critical to determining the knowledge base scope. Brainstorming determined all important and related nodes the chatbot needed to understand its function.
2. **Hierarchical Organization** The next phase saw the development of a logical hierarchy within the system. The use of "IS-A" relationships helped to structure the information taxonomy. Such a hierarchical system forms part of the fundamental characteristics of semantic networks (Collins & Quillian, 1969).
3. **Preliminary Sketching (Conceptualization)** The entire semantic network was sketched within the team's whiteboard space. This kind of "hands-on" activity helped to reveal relationships between nodes. Waterman (1986) makes note of how visualizing such concepts graphically helps to define relationships between various objects and processes prior to any coding effort.
4. **Digital Visualization (Formalization)** The next stage saw the formalization effort via Draw.io. This established the professional and editable version of the network. This phase mapped the general concept to a formal representation of the chatbot platform (Russell & Norvig, 2021).
5. **Validation and Approval** The final phase was establishing a process to confirm design validity – a peer review activity. The digital design was examined to ensure all logical aspects were sound and critical information visible. Once validated, the design was formally Approved for

implementation.

3. Implementation

The following chapter explains the development of the chatbot, from its concept to its actual implementation using Botpress (Botpress, 2024). During this process, the challenges that arose during the implementation will be discussed, as well as the solutions to those discussed challenges. Finally, the functions of the chatbot that were implemented will be discussed.

3.1 Challenges Faced

One of the first major challenges to be overcome during the implementation of the chatbot was the decision of in what nodes within the chatbot the AI functionality would be applied. Due to the design of the Aria chatbot itself, it was necessary to determine in which aspects of the chatbot the AI was required. For example, the AI would necessarily be required in nodes related to understanding the customer's intention with the chatbot, performing reasoning in response to those intentions, and generating friendly messages to greet or farewell the customers (Jurafsky & Martin, 2024).

Another of the major challenges to be overcome during the implementation of the chatbot was in the organization of the branches of the chatbot within Botpress. For example, any contradictions within the branches of the chatbot would lead to the creation of invalid nodes and customers becoming become confused during their interactions with the chatbot (Botpress, 2024). Furthermore, because the design of the chatbot incorporated user stories and flowcharts, it was also crucial to ensure that each branch of the chatbot led to a valid end state.

3.2 Solutions to Challenges

One of the solutions to the challenges of applying AI to the chatbot was to prepare a list of all of the nodes that would exist within the chatbot prior to implementation. Additionally, the AI functionality would only be applied to those individual nodes that would allow for the detection of the customer's intention, providing reasoning to those customers, and generating friendly messages to greet or farewell the customers (Chaves & Gerosa, 2021).

Additionally, the solution to the challenge of organizing the branches of the chatbot included organizing the nodes and branches prior to its deployment to Botpress. Furthermore, each branch was to be tested after its creation to ensure that the chatbot did not present any contradictions, errors, or invalid nodes for the customers (Botpress, 2024). As a result of this method of creating the chatbot, it became less likely that the customers would encounter errors with the chatbot, it became more reliable, and it was easier for the developers to manage the chatbot itself.

3.3 Key Functions Implemented

Several key functions of the chatbot were implemented into Aria. For example, the Aria chatbot implemented artificial intelligence to help with the detection of the customer's intended next step within the chatbot, performed reasoning in response to those steps, and generated friendly messages to greet or farewell the customers (Jurafsky & Martin, 2024; Chaves & Gerosa, 2021).

Second, the Aria chatbot implemented a knowledge base that functioned in a manner similar to semantic nets, with information about the airline and its services entered into the system in a way that could be easily understood by the chatbot and the customers, such as the relationship of the airline to its passengers, flights, tickets, payments, and airports (Rodriguez & Egenhofer, 2003).

Third, the Aria chatbot was able to store information regarding the customer within the chatbot, with the customer's permission, such that the information could be used to provide a more personalized experience for the customer (McTear et al., 2016).

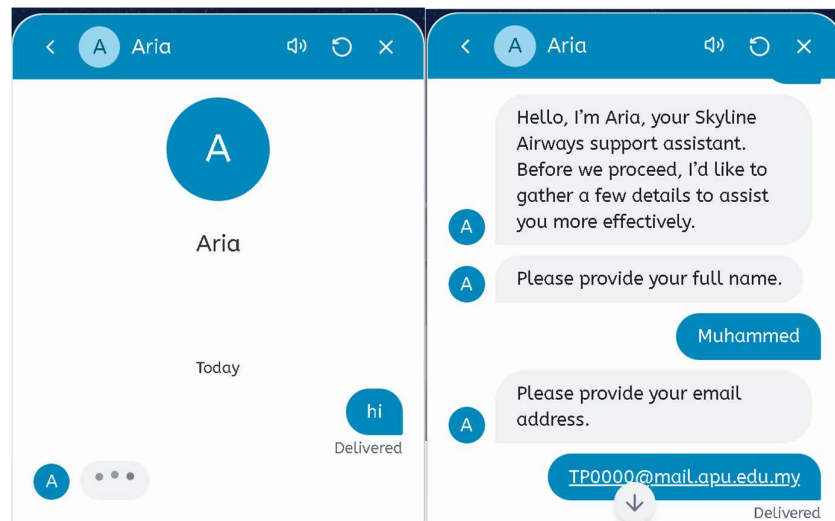
Finally, one of the main functions of the Aria chatbot was to utilize the pre-defined, node-based workflow of the chatbot to ensure that the customers experienced a reliable and error-free interaction with the airline.

3.4 Demonstration of Chatbot Features and Deployment

This additional section is included to demonstrate the implemented system to the readers in a further professional manner. The following discussion is of the various ways in which the chatbot accepts and responds to customers, performs its tasks, and is presented to the public through the airline's website.

3.5 Inputs and Outputs

1- Customer Details Collection & Main Menu



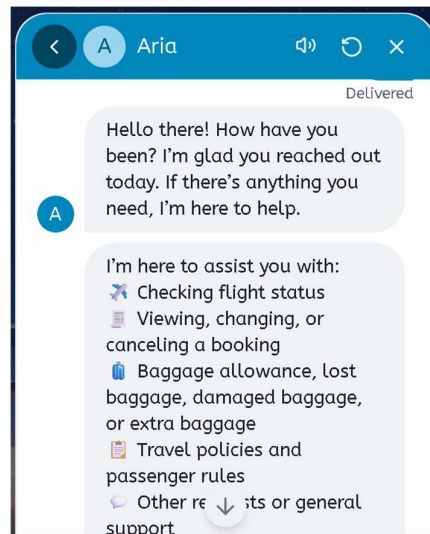


Fig 6

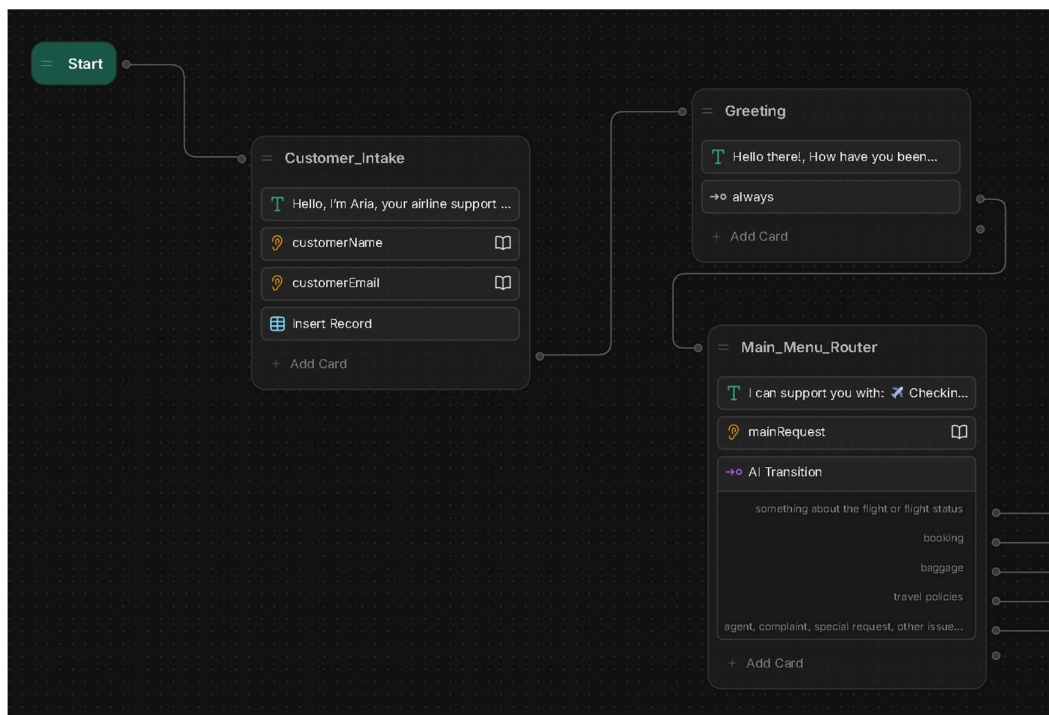


Fig 7

This is the workflow as implemented on Botpress. Users provide their name and email address, which get stored in a table. After receiving this data from the users, the bot greets them and sends them to the main routing component of the bot. This workflow indicates that the bot is structured in a way that allows for the users to provide data to the system, as well as interact with the bot in an easy-to-understand way.

The **Main_Menu_Router** component of the bot uses an AI Transition to determine which service the customer would like to use, such as flight status, booking, baggage, travel policies, or speaking to a live agent. The use of AI to route users to the different services indicates that this system was built with both

deterministic and AI elements working together to provide an efficient and effective experience for the users of the bot. Additionally, the use of slot filling to collect information from the users, storing that information in the database, and the use of AI to route those users to the correct services indicates an efficient and effective structure to the bot as a whole.

2- Flight Status Menu

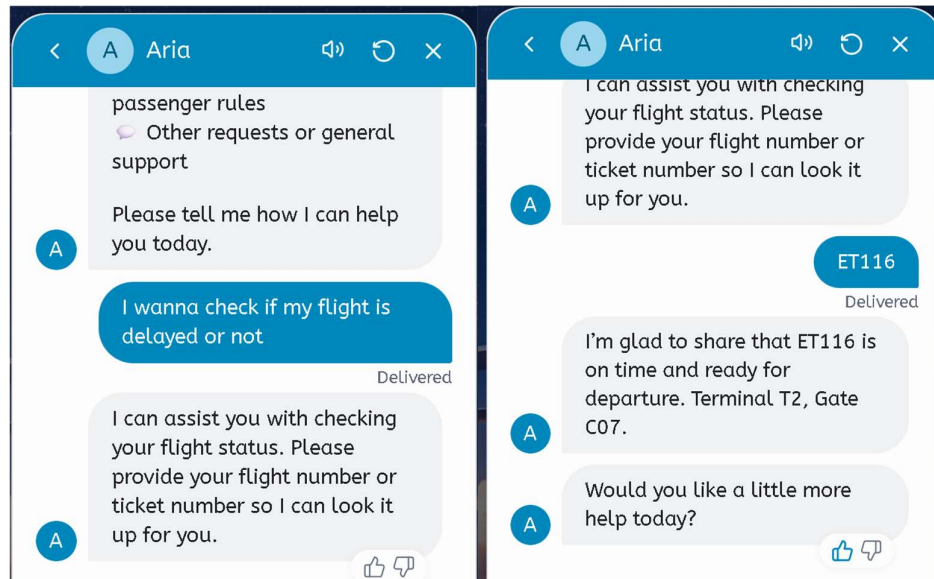


Fig 8. (input and output)

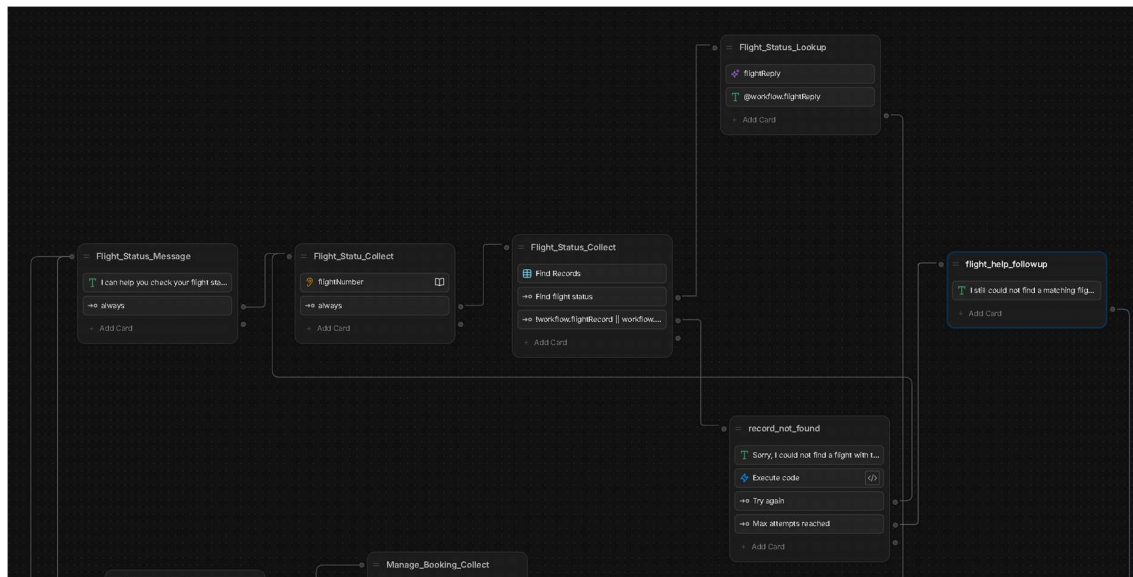


Fig 9

This flight status handling branch takes the user input for the flight number and looks up the flight by that number. If the system finds the flight with that number in its database, it responds to the user with information about that flight. If, however, the system does not find the flight with that number in its database, the system instead informs the user of that error and allows the user to attempt to enter the

flight number again (within a specified number of attempts). Finally, after this error-handling stage, the system can either direct the user to attempt to enter the flight number again, or it can allow the user to browse the other services offered by the airline. Thus, this branch handles flight information effectively and efficiently whilst providing a pleasant user experience overall.

3- Booking Management Menu

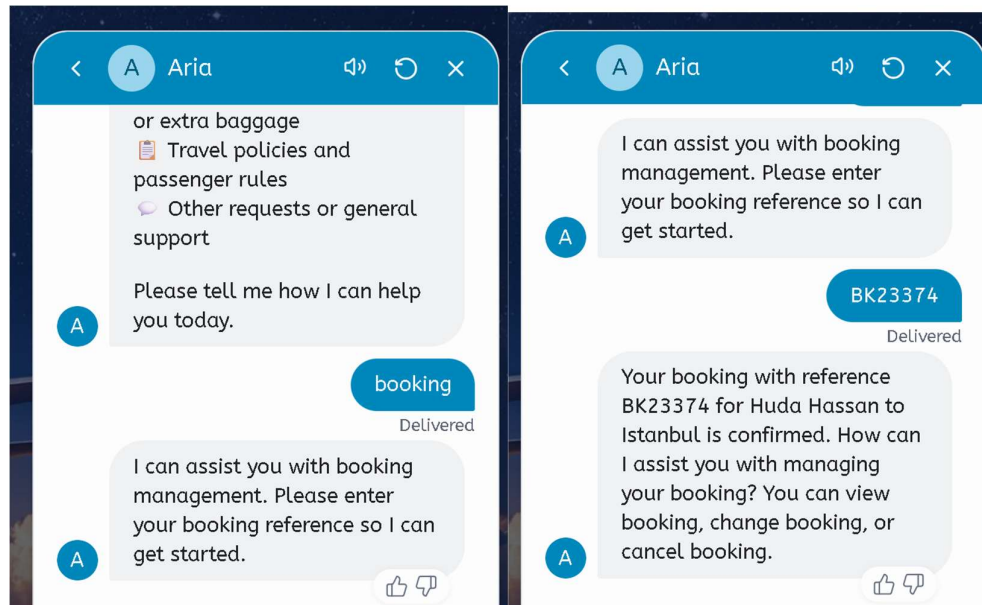


Fig 10

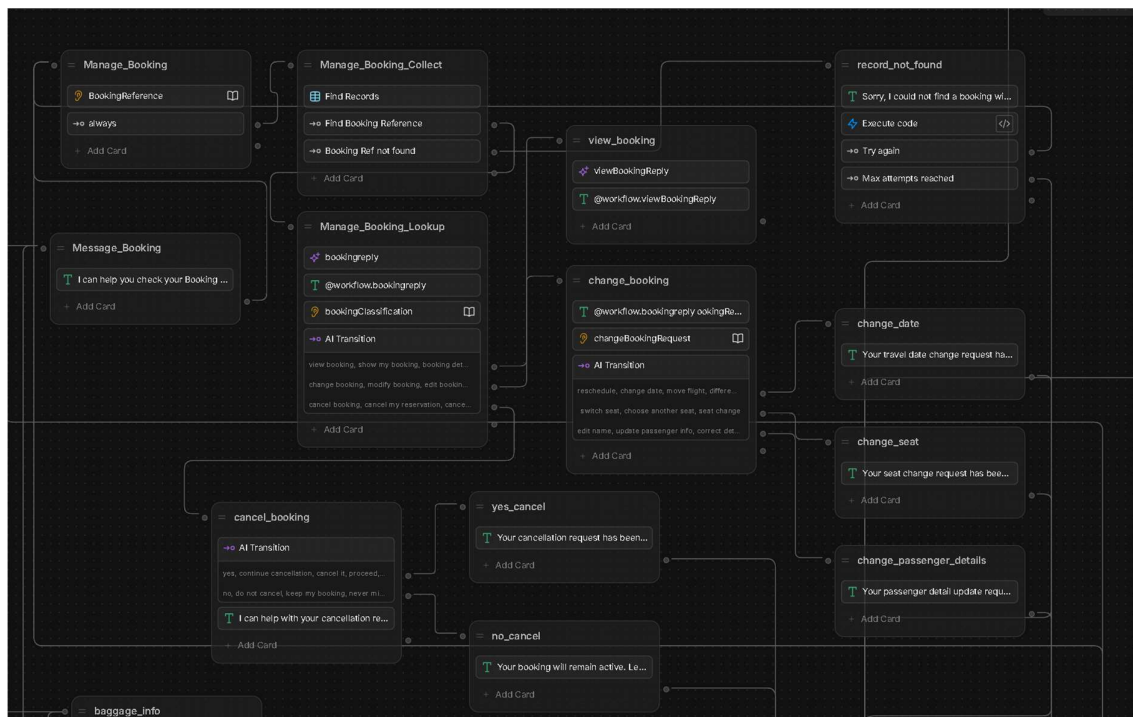


Fig 11

This workflow represents the booking management branch. After obtaining the booking reference from the user, the system can look up that booking record to retrieve the booking information. Once the booking has been validated, the system can determine the intent of the user regarding that booking. If the system finds the booking record by booking reference number, it can proceed to take the appropriate action for that user's selected intent regarding the booking.

Following validation of the booking, the system can route to several different sub-processes to assist the user with viewing, modifying or cancelling their booking. In each of these cases, the system will request additional information from the user and validate that the user intends to perform such an action. In the case of a cancellation request, the system will request confirmation from the user prior to taking any actions to cancel their booking. Thus, this branch of the workflow allows for the management of bookings after validation of the booking record.

4- Baggage Services Menu

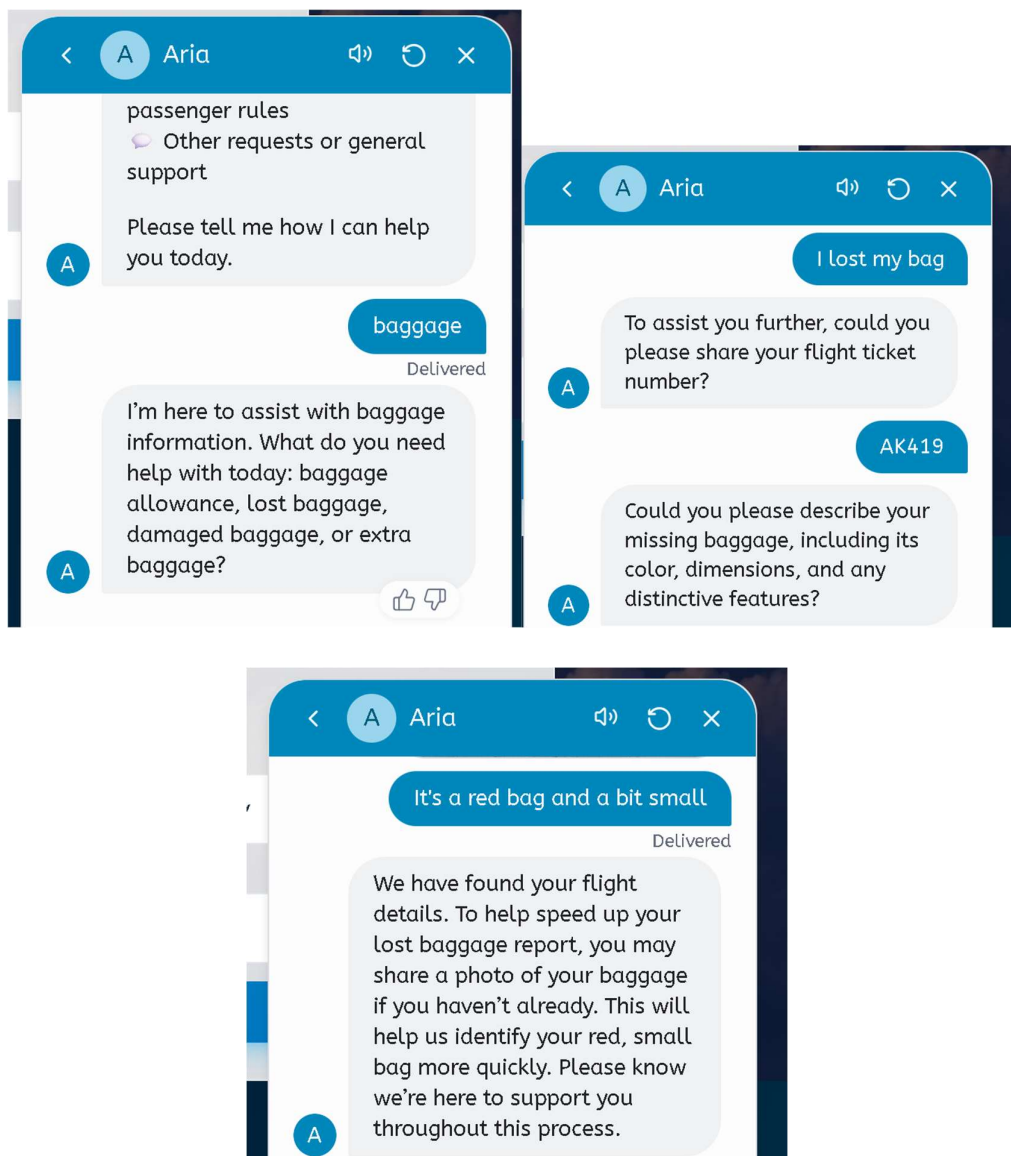


Fig 12

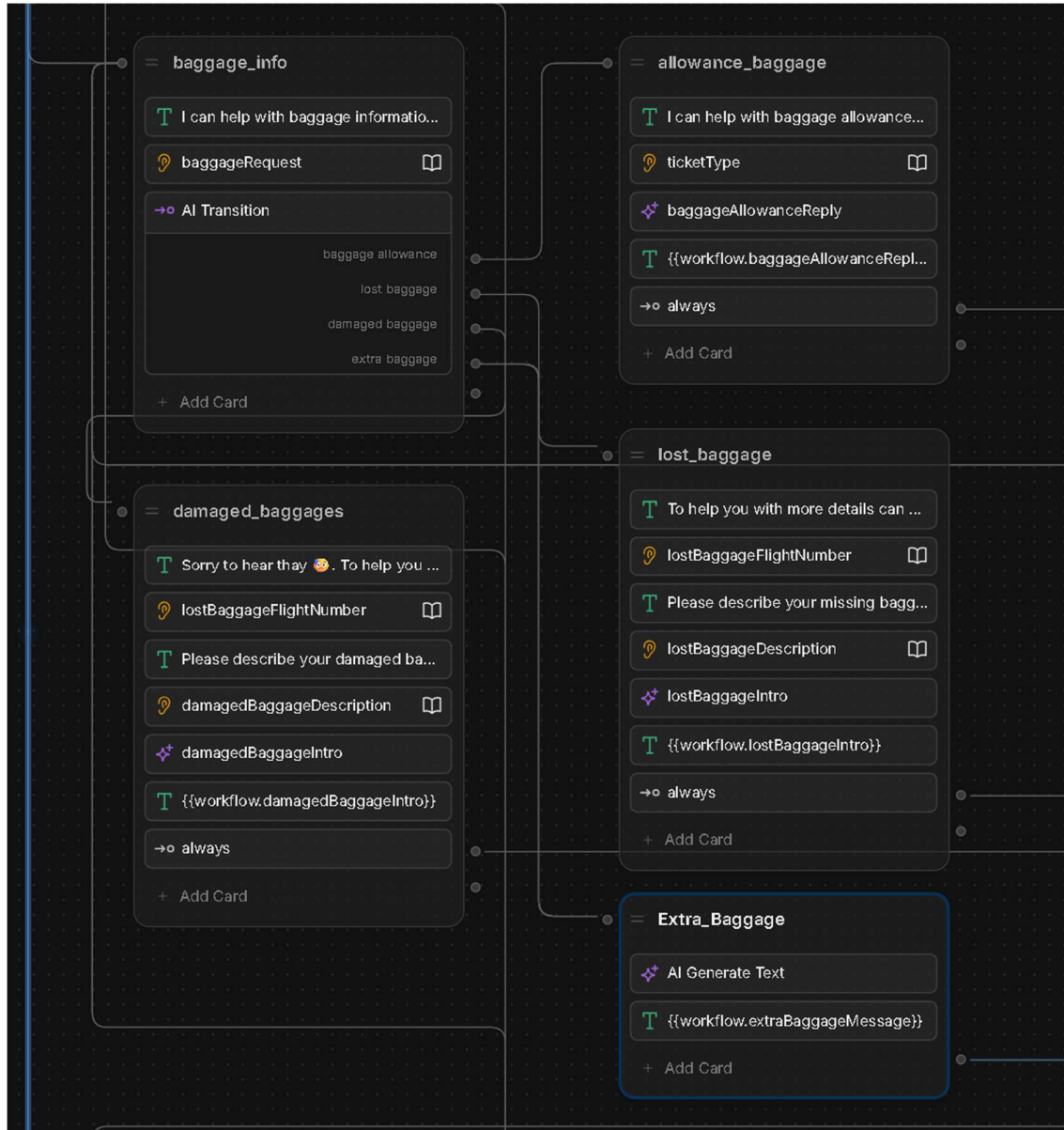


Fig 13

The following workflow represents the baggage support branch. This branch can receive multiple different types of requests regarding luggage. The workflow begins with a node that describes the type of request regarding baggage information that is to be received from the user, and which utilizes the AI transition to determine the type of request that the user has. Based on the user's request, the user is directed into one of several different categories, such as requests regarding baggage allowance, lost baggage, damaged baggage, or extra baggage. Each of these branches contains the steps and questions necessary to handle that type of request from the user, whether it requires the collection of information about the user's ticket type (as with questions regarding baggage allowance) or flight and baggage damage information (as with requests for lost or damaged baggage), or if it requires no inputs from the user other than the initiation of the conversation (as with requests for information about extra baggage). Thus, this workflow presents various paths for the responses to take based upon the different types of requests from the users.

5- Travel Policies Menu

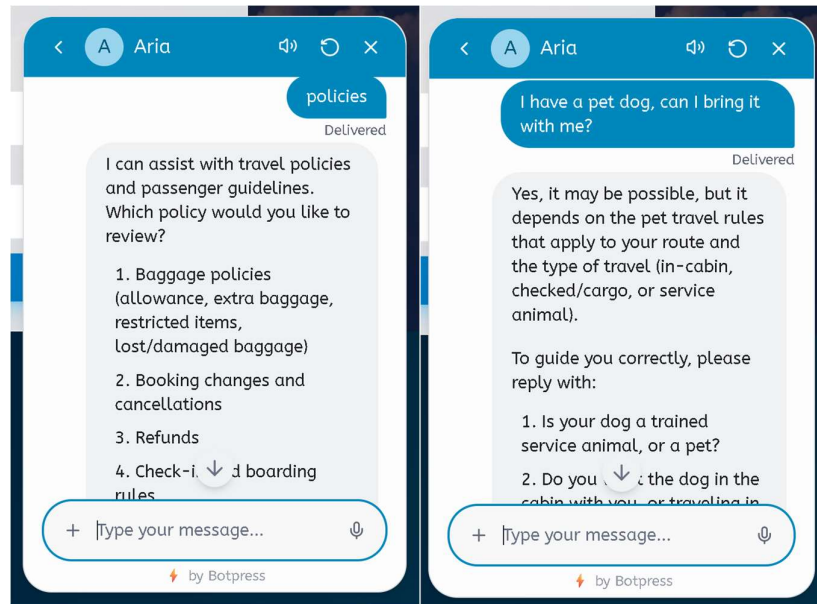


Fig 14

The travel policies node will use knowledge from the “Travel Policies and Passenger Compliance Rules” document located within the main knowledge base. The answer will be limited to only this one document, rather than potentially pulling from many different documents within the knowledge base. For the example illustrated, the travel policies node will first present the policies, and then answer the question regarding the travel of pets by referencing the policy itself, as well as potentially asking additional questions of the user to clarify their travel plans.

This type of node is significantly stronger than a standard node. A standard node will likely rely upon a set of static responses or retrieving knowledge from many different documents within the knowledge base. In contrast, the autonomous node functions in a way that is similar to a series of rules regarding the responses that the system should provide to the users questions. Thus, the autonomous node can provide answers that are more reliable and compliant with the guidelines set by the organization.

6- Other Help Menu

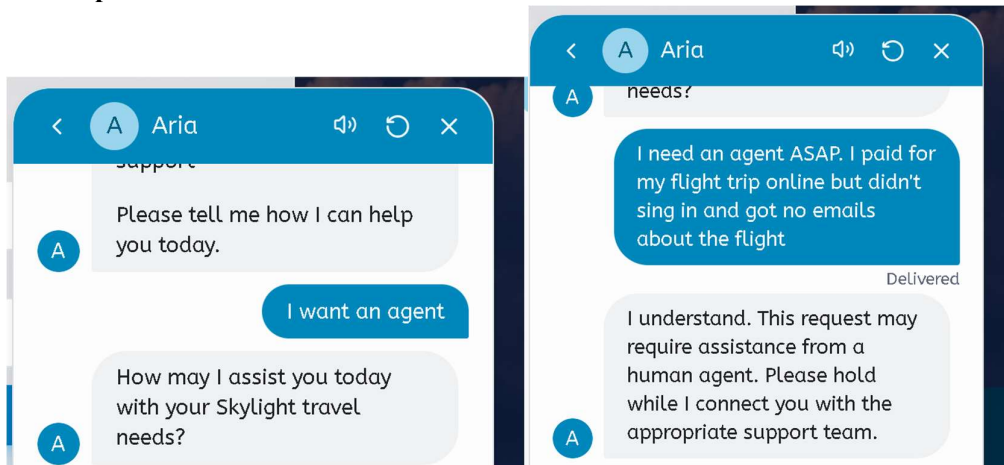


Fig 15

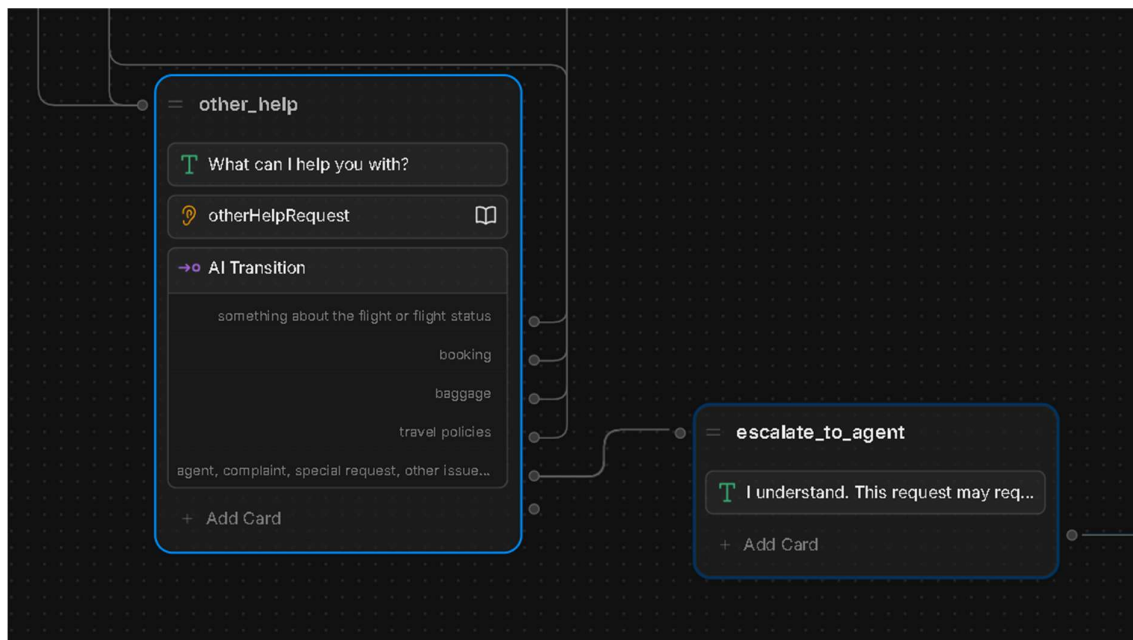


Fig 16

This section is for the human agent escalation. This will be used in the situation where the user's request cannot be fulfilled by the main service branches. The first node in this section will be similar to the main menu since it will classify the user's request to determine if it can be fulfilled by the existing features of the chatbot or if it requires the attention of a human agent. If the request can be fulfilled by the existing features of the chatbot, the user will be directed to those features. Otherwise, the user will be presented with a node that states that a human agent will be assisting with their request. This is conceptual only as it is technically challenging to build a system that directly connects to human agents to fulfill requests as this would drastically increase the complexity of the chatbot and move outside of the scope of this project.

3.6 Customer Navigation and Chatbot Features

Another feature that was added to the chatbot was the menu-based navigation with AI classification. Customers are presented with a menu of the service areas of the company, and the chatbot is able to direct their messages to the appropriate branch of the chatbot without the customer having to memorize the commands of the chatbot. Additionally, there are messages that ask if they would like to attempt to perform certain actions (such as booking actions, baggage actions, and messages that ask if the customer needs any more help with the airline). These features allow customers to navigate between the different services while minimizing the friction of such a process (Botpress, 2024).

3.7 Workflow Structure

The implementation of the chatbot utilizes one main Botpress workflow, with additional system workflows to handle errors, time-outs, and the ending of conversations. The main workflow contains 35 nodes in addition to flows for Error, Timeout, Conversation End, and a shared workflow. The main workflow contains nodes for the customer intake stage, routing, and follow-up stage, as well as branches for each of the task-specific workflows. Each of the structured tables has its own lookup workflow, while the travel policies exist as its own, separate node. Thus, the conversation was structured to utilize the determinism of the data with the flexibility provided by the travel policies.

3.8 Knowledge Bases Used

The chatbot uses a default knowledge-base collection composed of multiple airline support documents. These support documents contain information regarding the airline domain, such as the various concepts, relationships between those concepts, and the validation rules for those concepts. This type of knowledge base is best described as a semantic-network-oriented knowledge base. Airline domain concepts include objects such as Passenger, Booking, Ticket, Flight, Airport, Terminal, Gate, Payment, Refund, and Baggage, as well as more specific concepts like TravelDocument and ContactInfo. The relationship between these concepts is represented as Passenger -> Booking -> Ticket -> Flight -> Airport -> Terminal -> Gate. This type of relationship is characteristic of semantic networks. This type of knowledge base is not based upon frames, as the knowledge would be organized as frames in this instance, nor is it based upon production rules, as the information would be represented as IF-THEN rules in this scenario. Instead, this type of knowledge base is based upon the semantic relationships between the information, the workflow logic to access that information, and the table from which that information can be retrieved (Russell & Norvig, 2021; Rodriguez & Egenhofer, 2003).

Table 3. Knowledge-base documents used by Aria

Knowledge-base document	Role in the chatbot
Overview	Defines the scope, core service areas, and the menu-first / slot-filling operating principle.
Core Knowledge Model	Defines the major airline entities and the relationships among them.
Passenger, Contact, and Travel Documents	Governs verification and document-related support.
Booking, Payment, and Refund Support	Grounds booking, payment, and refund queries.
Ticket and Baggage Support	Grounds ticket, seat, and baggage-related responses.

Knowledge-base document	Role in the chatbot
Flight, Airport, Aircraft, Terminal, and Gate	Grounds flight-status and airport information.
Conversation Policy and Validation Rules	Defines safe dialogue sequence, clarification, and validation requirements.
Escalation Policy	Defines when the chatbot must hand the case to a human agent.
Semantic Relationships Representation	Explicitly represents the domain as nodes and relations, supporting the semantic-net interpretation.
Q&A / Retrieval Chunks	Provides retrieval-oriented question patterns and answer logic for common customer queries.
Travel Policies and Passenger Compliance Rules	Serves as the general policy source used by the autonomous travel-policies node, with specialized documents taking precedence when relevant.

Tables Used by the Chatbot

In addition to the knowledge-base documents are structured tables used by Aria for deterministic retrieval and storage of information. These tables are necessary to allow for the deterministic matching of information like airline identifiers to the correct objects in the knowledge base. Thus, the tables work in conjunction with the knowledge base documents to provide both semantic and deterministic knowledge to the system.

Structured tables used in the implemented chatbot

Table	Main fields	Operational purpose
CustomersTable (21 rows)	Name, Email	Stores intake information collected with customer permission at the start of the conversation.
BookingsTable (200 rows)	booking_reference, customer_name, destination, booking_status	Enables exact booking validation and supports view, change, and cancel actions.
FlightStatusTable (400 rows)	flight_number, status, departure_time, arrival_time, terminal, gate, destination	Supports exact flight-status lookups and operational responses.

3.9 Website Integration for Professional Presentation

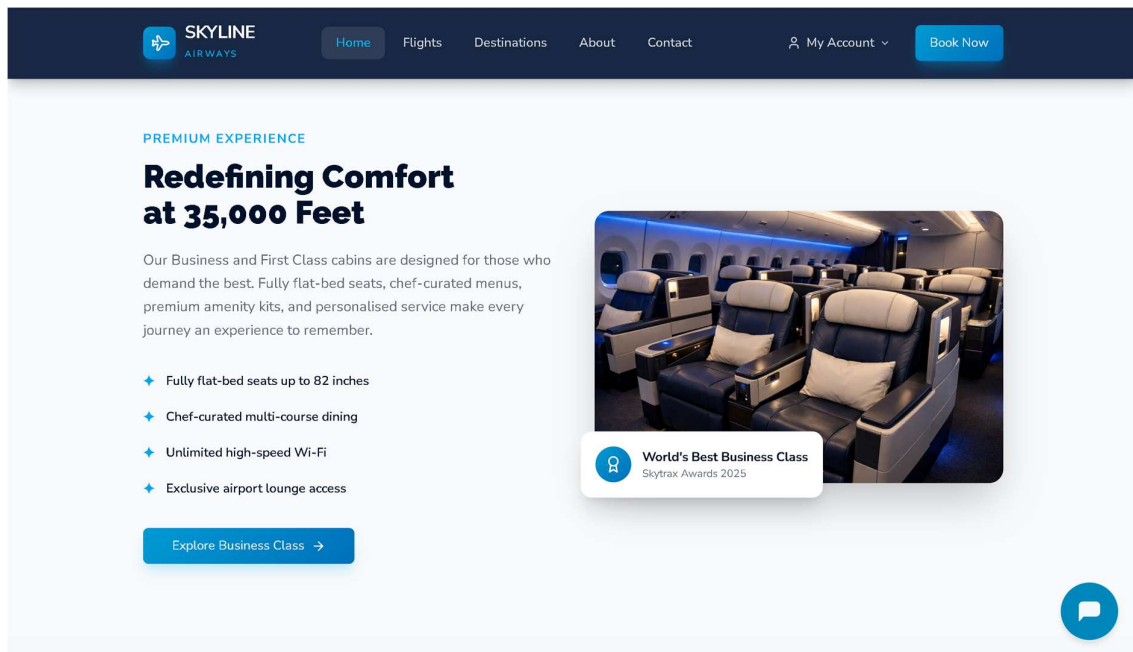


Fig 17

To further improve the professionalism and presentation quality of the chatbot, it was also deployed onto a public website for an airline company. The website features the chatbot as part of the airline company's website. For instance, the website's landing page includes elements of the airline company's branding and exposes users to areas of the website that relate to the airline's flights, flight statuses, booking systems, destinations, and airline features. Furthermore, the embedded chat within the airline's website allows for users to interact with the Aria chatbot. Thus, these additional deployment efforts help to showcase the chatbot as an integration into a public, customer-facing website that could be utilized by the company for commercial purposes (Skyline Airways, 2026).

4. Conclusion

The Aria chatbot development case study establishes that organized system design delivers effective support for airline customer service operations. The project achieved its goal by developing a functioning chatbot system that handles standard passenger requests through flight information and booking management and baggage details and travel assistance. The system establishes reliable user interaction through its low-code development platform and rule-based conversational framework, which handles user interaction according to established patterns of behavior.

The project used essential artificial intelligence principles to develop knowledge representation systems and dialogue management systems throughout its execution. The chatbot system applied semantic networks to construct a system that organizes and retrieves information, while the menu-based interaction system enhanced user experience through simplified access and reduced user errors. The combination of validation rules with structured workflows enabled accurate conversation tracking, which proved essential for airline operations since any incorrect information transmission caused **significant operational challenges**. The chatbot system has potential to improve customer service through its ability to provide immediate responses while reducing the requirement for human customer support staff. The airline receives operational benefits from this upgrade because it permits quicker service execution while decreasing expenses and enhancing resource distribution.

The present system contains multiple sections that require development enhancement work. The system needs updates, which must include natural language processing technology together with real-time airline database access and machine learning capabilities to improve system flexibility and personalized user experiences. The project demonstrates how organizations can build effective scalable chatbot systems by combining structured design elements with artificial intelligence concepts to achieve their business requirements.

5. References

- Russell, S., & Norvig, P. (2021). *Artificial intelligence: A modern approach* (4th ed.). Pearson.
- Peters, S. (2003, February). (PDF) *using semantic networks for knowledge representation in an intelligent environment*. Using semantic networks for knowledge representation in an intelligent environment.
https://www.researchgate.net/publication/2558448_Using_Semantic_Networks_for_Knowledge_Representation_in_an_Intelligent_Environment
- Holdsworth, J. (2025, November 18). *Benefits of chatbots*. IBM.
<https://www.ibm.com/think/insights/unlocking-the-power-of-chatbots-key-benefits-for-businesses-and-customers>
- International Organization for Standardization. (2019). *ISO 9241-210:2019 Ergonomics of human-system interaction — Part 210: Human-centered design for interactive systems*.
www.iso.org/standard/77520.html
- Nielsen, Jakob. “10 Usability Heuristics for User Interface Design.” *Nielsen Norman Group*, 30 Jan. 2024,
www.nngroup.com/articles/ten-usability-heuristics.
- Laubheimer, Page . “Preventing User Errors: Avoiding Unconscious Slips.” *Nielsen Norman Group*, 23 Aug. 2016,
www.nngroup.com/articles/slips.
- Nguyen, Quynh N., et al. “User Interactions with Chatbot Interfaces vs. Menu-Based Interfaces: An Empirical Study.” *Computers in Human Behavior*, vol. 128, Nov. 2021, p. 107093,
www.sciencedirect.com/science/article/pii/S0747563221004167
- Firdaus, Mauajama, et al. “A Multi-Task Hierarchical Approach for Intent Detection and Slot Filling.” *Knowledge-Based Systems*, vol. 183, 1 Nov. 2019, p. 104846,
www.sciencedirect.com/science/article/pii/S0950705119303211,
- Amna, Anis R., and Geert Poels. “Ambiguity in User Stories: A Systematic Literature Review.” *Information and Software Technology*, vol. 145, May 2022, p. 106824,
<https://doi.org/10.1016/j.infsof.2022.106824>.
- Chaves, A. P., & Gerosa, M. A. (2021). How should my chatbot interact? A survey on social characteristics in human–chatbot interaction design. *International Journal of Human–Computer Interaction*. <https://doi.org/10.1080/10447318.2020.1841438>
- Jurafsky, D., & Martin, J. H. (2024). *Speech and language processing* (3rd ed.). Stanford University. Retrieved March 22, 2026, from <https://web.stanford.edu/~jurafsky/slp3/>
- McTear, M., Callejas, Z., & Griol, D. (2016). *The conversational interface: Talking to smart devices*. Springer. <https://doi.org/10.1007/978-3-319-32967-3>

Rodriguez, M. A., & Egenhofer, M. J. (2003). Determining semantic similarity among entity classes from different ontologies. *IEEE Transactions on Knowledge and Data Engineering*, *15*(2), 442–456. <https://doi.org/10.1109/TKDE.2003.1185844>